Ministerul Educaţiei și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

RAPORT

Lucrarea de laborator nr.2

la Dispozitive electronice și mijloace tehnice de protecție a informației

Achiziția semnalului și condiționarea software

A efectuat: **gr. SI-251, Sîromeatnicov Vlad**

A verificat: **Astafi Valentina asist. univ.**

Chişinău, 2026

CUPRINS

LISTA DE ABREVIERI ȘI DEFINIȚII 3

FIȘĂ DE LABORATOR 4

INTRODUCERE 5

1 ACHIZIȚIA ȘI CONDIȚIONAREA SOFTWARE A SEMNALULUI 6

1.1 Achiziția semnalului digital 6

1.2 Perturbații ale semnalului 6

1.3 Saturarea valorilor 6

1.4 Filtrarea mediană și filtrarea prin mediere 6

1.5 Conversia în mărime fizică 7

2 PROIECTAREA SISTEMULUI PE ESP32 8

2.1 Circuitul hardware simplificat 8

2.2 Achiziția periodică cu FreeRTOS 9

2.3 Arhitectura funcțională și lanțul de condiționare software 9

3 IMPLEMENTAREA SOFTWARE 11

3.1 Arhitectura sistemului în mediul FreeRTOS 11

3.2 Structura modulară a lanțului de prelucrare 11

4 TESTARE ȘI ANALIZA REZULTATELOR 12

4.1 Metodologia de testare 12

4.2 Compararea seriei brute cu seria condiționată 12

CONCLUZII 15

NOTĂ PRIVIND UTILIZAREA INSTRUMENTELOR AI 16

BIBLIOGRAFIE 17

ANEXE 18

LISTA DE ABREVIERI ȘI DEFINIȚII

A – indice de activitate, mărimea fizică finală, în procente din plaja ieșirii senzorului

GND (eng. Ground) – linie de masă

LED (eng. Light-Emitting Diode) – diodă electroluminiscentă

NPN – tip de tranzistor bipolar

R (eng. Resistor) – rezistor

V_HIGH – tensiunea nivelului HIGH al ieșirii senzorului

ESP32 – microcontroler cu Wi-Fi integrat, produs de Espressif

FreeRTOS – sistem de operare în timp real pentru microcontrolere

GPIO (eng. General Purpose Input/Output) – pin de uz general

HC-SR501 – modul PIR analizat în lucrarea de laborator nr.1

HIGH / LOW – nivel logic ridicat / nivel logic scăzut

PIR (eng. Passive Infrared) – senzor pasiv de infraroșu

PlatformIO – mediu de dezvoltare pentru sisteme embedded

STDIO (eng. Standard Input/Output) – fluxuri standard de intrare și ieșire, redirecționate aici către portul serial

Task – fir de execuție gestionat de planificatorul FreeRTOS

Timestamp – marcă temporală a citirii

Use case – caz de utilizare

Wokwi – simulator online pentru plăci ESP32

FIȘĂ DE LABORATOR

Scopul lucrării

Lucrarea are ca scop formarea deprinderilor de achiziție și condiționare software a semnalului provenit de la un senzor fizic, într-un sistem embedded multitasking bazat pe microcontroler. Ieșirea prototipului minimal din L1 este preluată de un ESP32 și transformată, printr-un lanț de prelucrări software executat într-un task FreeRTOS, într-o mărime fizică utilizabilă și de încredere.

Obiective

- înțelegerea importanței eșantionării periodice, deterministe, în sisteme embedded multitasking;
- recunoașterea surselor de perturbație a semnalului unui senzor PIR;
- aplicarea și diferențierea saturării, a filtrului median și a filtrului de mediere;
- urmărirea transformării valorii brute în mărimea fizică finală;
- compararea seriei brute cu cea condiționată, în prezența unei perturbații introduse deliberat;
- conștientizarea legăturii dintre calitatea semnalului la sursă și încrederea în datele din etapele ulterioare.

Use case

Dispozitiv electronic amplasat în interiorul unei locuințe sau al unui apartament, conectat la rețeaua Wi-Fi domestică, partajată cu celelalte dispozitive personale din casă, destinat detectării mișcării și trimiterii unei notificări, eventual însoțite de o fotografie – caz tipic pentru sistemele de alarmă și soneriile video inteligente.

INTRODUCERE

Un detector de mișcare transformă radiația infraroșie a mediului într-un semnal electric preluat de un microcontroler. Semnalul nu este însă întotdeauna stabil: pot apărea declanșări false, goluri scurte în timpul unei detecții sau impulsuri produse de alimentare ori de cablaj. Dacă sistemul interpretează direct aceste variații ca schimbări reale, trimite notificări false.

Lucrarea nr.2 continuă prototipul minimal din L1: un modul PIR echivalent cu HC-SR501, urmat de un etaj cu tranzistor NPN care comandă un LED. Ieșirea digitală OUT a modulului este citită de un pin al ESP32 și prelucrată succesiv prin saturare, filtru median, filtru de mediere și conversie, iar rezultatul este raportat prin portul serial.

Filtrarea software nu poate repara o interfațare electrică incorectă. Intrările ESP32 funcționează la 3,3 V. În schema finală folosită în Wokwi, ieșirea digitală OUT este conectată direct la GPIO4, fără divizor 1 kΩ/2 kΩ, deoarece nivelul HIGH al semnalului simulat este compatibil cu intrarea de 3,3 V. Rezistorul R1 = 4,7 kΩ formează o ramură separată între OUT și baza tranzistorului NPN, pentru comanda LED-ului. Pentru un modul fizic a cărui ieșire OUT poate ajunge la aproximativ 5 V, trebuie verificat nivelul înainte de conectarea la ESP32; dacă este necesar, se folosește un divizor 1 kΩ în serie și 2 kΩ către GND, dimensionat astfel încât GPIO4 să nu depășească 3,3 V. Rezultatele capitolului 4 de mai jos sunt cele furnizate de rularea în Wokwi.

1 ACHIZIȚIA ȘI CONDIȚIONAREA SOFTWARE A SEMNALULUI

Detectarea mișcării în locuință se bazează pe senzori PIR, sensibili la variația radiației infraroșii emise de corpul uman. Modulul conține deja amplificarea, comparatorul și temporizarea, deci software-ul primește un semnal digital deja format, dar nu și lipsit de perturbații.

1.1 Achiziția semnalului digital

Ieșirea OUT a modulului are două niveluri: LOW (0 V) în repaus și HIGH (3,3 V [1]) la detecție. Interfața preluată în L2 este digitală, deci citirea se face cu digitalRead() pe un pin GPIO, iar valoarea brută este 0 sau 1; nu este necesar convertorul analog-digital. Citirea este periodică, la momente fixe, pentru ca filtrele să opereze pe eșantioane echidistante.

1.2 Perturbații ale semnalului

Semnalul brut poate fi afectat de impulsuri electrice induse de alimentare, cablaj sau comutarea tranzistorului, de goluri scurte în timpul unei detecții și de alarme false ale senzorului. Conform fișei tehnice HC-SR501 [1], timpul de menținere este reglabil între 0,5 și 200 s. Prin urmare, o declanșare falsă reală cauzată de un curent de aer sau de o sursă de căldură poate menține ieșirea HIGH cel puțin 0,5 s, adică minimum cinci eșantioane la Ts = 100 ms; un astfel de semnal trece de filtrul median N = 5 și nu poate fi eliminat de acesta ca un glitch izolat. Filtrul median din această lucrare vizează doar glitch-urile electrice scurte, nu alarmele false ale senzorului. Pentru acestea se poate adăuga o confirmare prin două detecții într-un interval definit sau o durată minimă de menținere a stării HIGH înainte de declararea mișcării.

1.3 Saturarea valorilor

Saturarea limitează o valoare la un interval admis: sub limita inferioară se folosește limita inferioară, iar peste limita superioară se folosește limita superioară. Pentru intrarea digitală, intervalul este 0…1, corespunzător nivelurilor LOW și HIGH. În funcționare normală, digitalRead() întoarce numai LOW sau HIGH (0 sau 1), deci valoarea 7 nu poate proveni direct din digitalRead(); perturbația 7 din test este un caz artificial injectat în cod pentru a verifica funcția saturate(). Saturarea software nu protejează pinul împotriva supratensiunii și nu înlocuiește adaptarea electrică a nivelului.

1.4 Filtrarea mediană și filtrarea prin mediere

Filtrul median păstrează o fereastră de N_{MED} eșantioane, le ordonează și returnează valoarea din mijloc, conform relației (1.1):

$$y_{med}[n] = \text{mediana}\{\, x[n], x[n-1], \ldots, x[n-N_{MED}+1] \,\} \tag{1.1}$$

unde x[n] este eșantionul saturat, iar N_{MED} – fereastra filtrului, număr impar. Pentru $N_{MED} = 5$, valorile {0, 0, 1, 0, 0} devin, după sortare, {0, 0, 0, 0, 1}, iar mediana este 0: impulsul izolat nu influențează rezultatul. Pentru un semnal binar, medianul este votul majorității și elimină orice grup de cel mult $(N_{MED} - 1) / 2$ eșantioane consecutive, adică 2 pentru $N_{MED} = 5$. O fereastră mică răspunde rapid, dar elimină mai puține perturbații; una mare întârzie detectarea unei schimbări reale.

Filtrul de mediere calculează media aritmetică a ultimelor N_{AVG} valori de la ieșirea filtrului median, conform relației (1.2):

$$y_{avg}[n] = (1 / N_{AVG}) \cdot \Sigma\, y_{med}[n - i],\ \ i = 0 \ldots N_{AVG} - 1 \tag{1.2}$$

Pentru un semnal binar, media este ponderea eșantioanelor HIGH din fereastră, deci o valoare între 0 și 1. Ea atenuează perturbațiile rămase după median, dar netezește și întârzie fronturile reale. Cele două filtre au roluri diferite: mediana elimină impulsurile izolate, media reduce efectul celor rămase. Ordinea folosită este: saturare → mediană → mediere → conversie.

1.5 Conversia în mărime fizică

Valoarea filtrată yavg este convertită în indicele de activitate A, în procente, conform relației (1.3). În raportul Serial Monitor este păstrată și coloana V [mV], calculată convențional prin V = yavg · 3300 mV. Aceasta nu este o măsurare analogică a ieșirii, ci o estimare echivalentă a nivelului mediu al unui semnal binar: de exemplu, avg = 0,5 corespunde unui nivel HIGH în 50% din fereastra de mediere și unei valori echivalente V = 1650 mV. Parametrul decizional este A, iar starea „MISCARE” se declară pentru A ≥ 50%. Prin urmare, V nu are semnificație de măsurare: este doar avg exprimat în mV, util pentru a compara nivelul mediu cu pragul logic de 3,3 V; decizia se ia exclusiv pe A.

$$A = 100 \cdot (y_{avg} - LVL_{MIN}) / (LVL_{MAX} - LVL_{MIN}) \tag{1.3}$$

$$V = A / 100 \cdot V_{HIGH} \tag{1.4}$$

unde $LVL_{MIN} = 0$ și $LVL_{MAX} = 1$ sunt limitele de saturare, iar $V_{HIGH} = 3300$ mV. Starea „mișcare” se declară pentru A ≥ 50 %. Pentru un senzor binar nu există o curbă de calibrare analogică; singurul parametru de ales este pragul de decizie, fixat la jumătate din plajă.

2 PROIECTAREA SISTEMULUI PE ESP32

ESP32 este un microcontroler cu două nuclee și Wi-Fi integrat, potrivit dispozitivelor IoT, iar sistemul de operare FreeRTOS permite execuția deterministă a task-urilor. Platforma asigură atât achiziția periodică, cât și lanțul de condiționare software, iar Wi-Fi-ul este necesar ulterior pentru trimiterea notificărilor.

2.1 Circuitul hardware simplificat

Circuitul este prototipul minimal din L1 (figura 2.1): modulul PIR1 alimentat la 5 V, rezistorul R1 = 4,7 kΩ între ieșirea OUT și baza tranzistorului NPN TQ1, LED-ul D1 și rezistorul R2 = 330 Ω în colector. Față de L1, placa Arduino UNO este înlocuită de ESP32, care furnizează alimentarea de 5 V (pinul VIN), iar ieșirea OUT se conectează suplimentar la GPIO4, ales pentru că nu are restricții de pornire. Etajul cu tranzistor rămâne neschimbat: la 3,3 V curentul de bază este (3,3 − 0,7) / 4,7 kΩ ≈ 0,55 mA, suficient pentru a satura tranzistorul la un curent de colector de circa 8,5 mA.

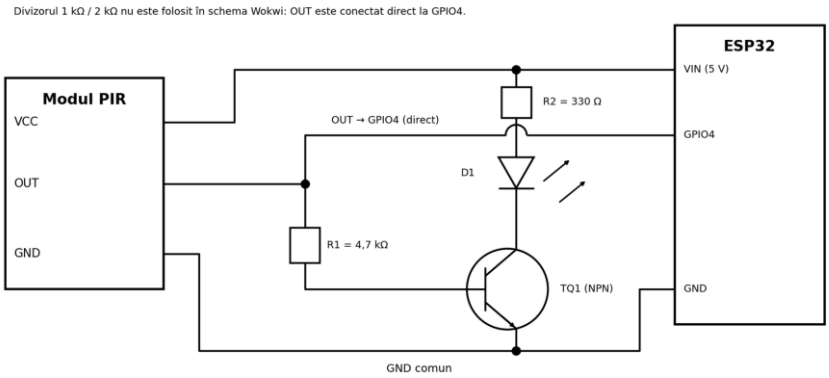


Figura 2.1 – Schema electrică revizuită a prototipului minimal cu ESP32

Ramificarea ieșirii OUT nu modifică funcționarea etajului din L1: LED-ul D1 se aprinde cât timp OUT este HIGH, iar ESP32 doar observă nivelul acestei ieșiri. Singura legătură nouă față de L1 este firul de la OUT la GPIO4.

În schema finală simulată în Wokwi, divizorul 1 kΩ/2 kΩ nu este montat: OUT se conectează direct la GPIO4, iar ramura R1 către baza tranzistorului este separată. Pentru un modul fizic cu OUT de 5 V, compatibilitatea electrică trebuie verificată înainte de conectare. Conexiunile dintre modul, tranzistor și placă sunt date în tabelul 2.1.

Tabelul 2.1 – Conexiunile prototipului la placa ESP32

Conexiune	Detalii
PIR1: VCC	VIN (5 V) al plăcii ESP32
PIR1: GND	GND al plăcii ESP32; masă comună cu emitorul TQ1

Continuare Tabelul 2.1

Conexiune	Detalii
PIR1: OUT	GPIO4 (intrare digitală, conectare directă în schema Wokwi) și, separat, R1 = 4,7 kΩ către baza lui TQ1
GPIO4 / divizor	Divizorul 1 kΩ/2 kΩ nu este folosit în schema finală Wokwi; se adaugă numai dacă nivelul OUT al unui modul fizic depășește limita admisă de 3,3 V
TQ1 (NPN)	Emitor la GND; colector către LED-ul D1
D1 + R2 = 330 Ω	În serie, în ramura colectorului lui TQ1, către 5 V

Modulul, placa și emitorul lui TQ1 au masă comună, deci semnalul de la OUT are o singură referință. Pinul GPIO4 este o intrare simplă, fără rol special la pornirea plăcii, iar rezistorul R1 limitează curentul de bază al tranzistorului.

2.2 Achiziția periodică cu FreeRTOS

Aplicația are un singur task de achiziție, care citește senzorul, aplică etapele de condiționare, convertește valoarea și raportează rezultatul prin STDIO. Perioada constantă se obține cu vTaskDelayUntil(), care trezește task-ul la momente absolute de timp [2], spre deosebire de vTaskDelay(), care așteaptă intervalul cerut după apel și acumulează variații ale perioadei. Perioada este T_s = 100 ms (10 eșantioane pe secundă), suficient de mică pentru a urmări fronturile și mult mai mare decât durata prelucrărilor. Timpul este furnizat în milisecunde de funcția millis(). Parametrii sunt constante în cod (tabelul 2.2).

Tabelul 2.2 – Parametrii configurabili

Constantă	Valoare	Justificare
TS_MS	100 ms	perturbațiile durează câteva eșantioane, semnalul util – secunde
LVL_MIN / LVL_MAX	0 / 1	nivelurile LOW / HIGH din fișa tehnică [1]
N_MED	5	elimină impulsuri de cel mult 2 eșantioane (200 ms)
N_AVG	10	media pe o secundă, indice A în trepte de 10 %
A_PRAG	50 %	jumătate din plaja ieșirii

Toți parametrii din tabelul 2.2 sunt definiți ca constante la începutul programului. Modificarea unei ferestre sau a pragului cere doar schimbarea valorii constantei, nu a funcțiilor de calcul.

2.3 Arhitectura funcțională și lanțul de condiționare software

Lanțul este executat periodic în task-ul de achiziție, iar fiecare etapă din figura 2.2 are un rol definit. Ieșirea OUT a modulului este citită digital de GPIO4 (0 sau 1). Valoarea este limitată de saturare la 0…1, apoi filtrul median (N_{MED} = 5) elimină impulsurile izolate, iar filtrul de mediere (N_{AVG} = 10) calculează

ponderea eșantioanelor HIGH din ultima secundă. Conversia dă indicele A și valoarea echivalentă V (în mV), iar raportarea prin STDIO afișează valoarea brută și valorile condiționate, cu marcă temporală.

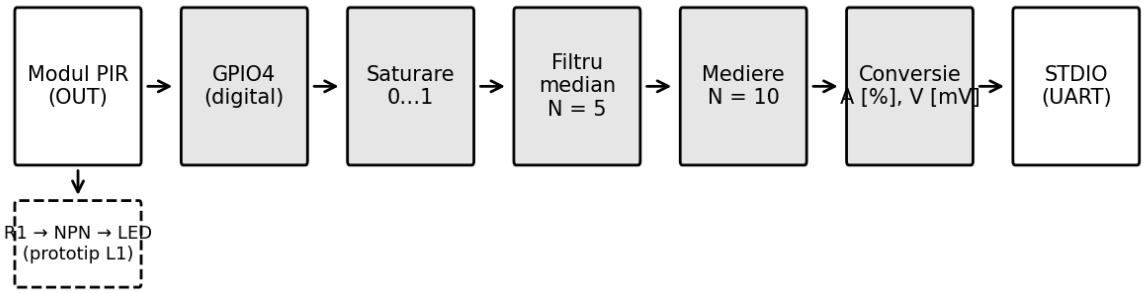


Figura 2.2 – Lanțul software de achiziție și condiționare a semnalului

Figura 2.2 arată că fiecare etapă primește doar ieșirea celei precedente, deci etapele pot fi modificate independent. Întârzierea de grup a lanțului se calculează cu relația (2.1):

$$\tau \approx ((N_{MED} - 1) / 2 + (N_{AVG} - 1) / 2) \cdot T_s \qquad (2.1)$$

Pentru valorile alese, $\tau = (2 + 4{,}5) \cdot 100$ ms = 650 ms, acceptabilă pentru o notificare de mișcare.

3 IMPLEMENTAREA SOFTWARE

Implementarea transpune lanțul într-o aplicație PlatformIO pentru placa esp32dev, cu framework-ul Arduino; codul complet este în anexa A, iar configurația proiectului în anexa B.

3.1 Arhitectura sistemului în mediul FreeRTOS

Task-ul de achiziție este creat în setup() pe nucleul 1, iar loop() își șterge propriul task. Task-ul memorează momentul de trezire în xLastWake și apelează vTaskDelayUntil() cu perioada de 100 ms, deci eroarea de temporizare nu se acumulează, spre deosebire de delay(). Imediat după trezire se citește timpul cu millis(), se execută lanțul și se afișează cu printf() o linie cu marca temporală, valoarea brută, valorile după fiecare etapă, indicele A, valoarea echivalentă V și starea:

```
for (;;) {
    vTaskDelayUntil(&xLastWake, xPeriod);
    uint32_t t = millis();
    int   raw = readRaw();
    int   sat = saturate(raw);
    int   med = medianFilter(sat);
    float avg = averageFilter(med);
    float act = averageToActivity(avg);
    printf(...);
}
```

3.2 Structura modulară a lanțului de prelucrare

Fiecare etapă este o funcție separată (tabelul 3.1), iar task-ul doar le apelează în ordine. Ferestrele filtrelor sunt constantele N_MED și N_AVG, iar buffer-ele circulare se inițializează cu primul eșantion, pentru a evita o tranziție falsă la pornire. Funcția injectDisturbance(), activă doar pentru SIM_DISTURBANCE = 1, suprapune periodic perturbații de test (în tabelul 4.1 sunt cele din fragmentul Wokwi analizat); în exploatare constanta se setează la 0.

Tabelul 3.1 – Funcțiile aplicației

Funcție	Rol
readRaw()	citește pinul cu digitalRead(), fără nicio prelucrare
saturate()	limitează valoarea la 0…1
medianFilter()	filtru median pe fereastră alunecătoare de N_MED eșantioane
averageFilter()	medie aritmetică pe fereastră alunecătoare de N_AVG valori
averageToActivity() / activityToMillivolts()	conversia în indice de activitate A și în valoarea echivalentă V
injectDisturbance()	perturbații de test, active doar dacă SIM_DISTURBANCE = 1
taskAcquire()	task FreeRTOS: temporizare, apelul lanțului, raportare prin STDIO

Separarea în funcții permite verificarea fiecărei etape în parte, iar înlocuirea unui filtru nu afectează celelalte etape. Singura funcție care depinde de hardware este readRaw(), deci portarea pe alt senzor sau alt pin modifică un singur loc din cod.

4 TESTARE ȘI ANALIZA REZULTATELOR

Testarea urmărește comportamentul lanțului de achiziție și condiționare în rularea Wokwi, folosind valorile afișate de Serial Monitor la intervale de aproximativ 100 ms. Se urmăresc eliminarea impulsurilor izolate, răspunsul la frontul crescător al semnalului și întârzierea introdusă de filtre.

4.1 Metodologia de testare

Testul prezentat în această secțiune provine din rularea proiectului ESP32 în Wokwi, cu intrarea OUT simulată prin senzorul PIR din Wokwi, conectat la GPIO4. Datele din Serial Monitor au timestamp-uri la pas de 100 ms și includ valorile brut, sat, med, avg, A, V și starea (figura 4.2 și tabelul 4.2). Impulsul brut de la 1601 ms și valoarea brută 7 de la 2101 ms sunt perturbații injectate software prin injectDisturbance() (SIM_DISTURBANCE = 1), nu semnale provenite de la senzor; valoarea 7 este un test artificial al saturării, nu o valoare pe care digitalRead() ar putea-o întoarce în funcționare normală. Figura 4.1 este o reprezentare grafică a acelorași date, construită cu un script Python (anexa C), și nu o măsurare independentă.

Tabelul 4.1 – Perturbațiile introduse și efectul lor

Nr.	Observație / test	Moment, ms	Efect observat în Wokwi
1	Impuls brut HIGH scurt	1601	brut = 1, sat = 1, med = 0, A = 0%; starea rămâne repaus
2	Valoare artificială 7 pentru testul saturării	2101	brut = 7, sat = 1; 7 este injectat în test, nu poate fi returnat de digitalRead(); fiind înconjurat de eșantioane HIGH, nu schimbă med
3	Front crescător real	1901–2401	med devine 1 la 2001 ms, A = 50% la 2401 ms; starea devine MISCARE

Perturbațiile injectate în software sunt teste artificiale pentru validarea etapelor de condiționare. Ele nu reproduc fidel o alarmă falsă reală a modulului HC-SR501, deoarece timpul său de menținere este de cel puțin 0,5 s. Filtrarea mediană N = 5 este destinată glitch-urilor electrice de cel mult două eșantioane; pentru alarme false persistente este necesară o logică suplimentară de confirmare.

4.2 Compararea seriei brute cu seria condiționată

Datele din Serial Monitor arată evoluția valorii brute și a etapelor de condiționare. În repaus, avg, A și V sunt zero după stabilizarea filtrelor. La frontul crescător, med devine 1 la 2001 ms, A ajunge la pragul de 50% la 2401 ms și starea se schimbă în „MISCARE". Tabelul 4.2 prezintă eșantioane

reprezentative din captura Wokwi (figura 4.2). Figura 4.1 reprezintă grafic aceleași date și a fost construită cu scriptul din anexa C.

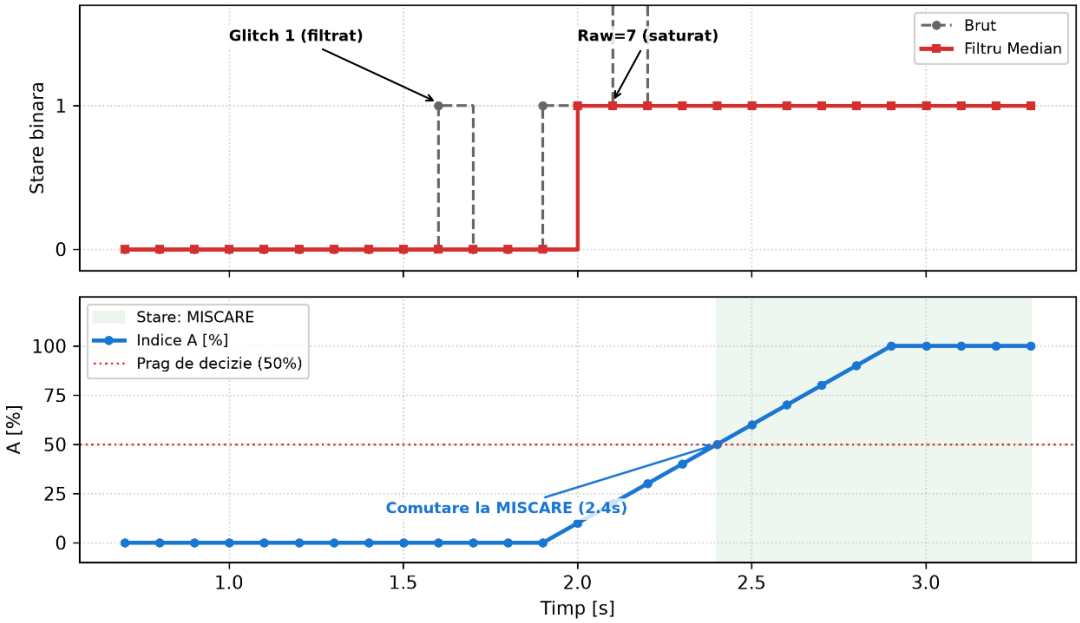


Figura 4.1 – Reprezentarea grafică a seriei brute și condiționate, construită în Python pe baza datelor din Wokwi

Figura 4.1 a fost construită printr-un script Python cu biblioteca matplotlib (anexa C), care a folosit ca referință datele din figura 4.2 și tabelul 4.2. Ea arată că ieșirea medianului elimină impulsul izolat de la 1,6 s și păstrează frontul real, iar valoarea brută 7 este limitată la 1. Indicele A crește în trepte de 10 % pe durata unei secunde și atinge pragul de 50 % la 2,4 s, ca în datele din Wokwi.

Un fragment din raportul STDIO, la frontul crescător, este prezentat în figura 4.2.

```
  701 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
  801 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
  901 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1001 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1101 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1201 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1301 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1401 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1501 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1601 |   1 |   1 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1701 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1801 |   0 |   0 |   0 |  0.00 |    0.0 |     0.0 | repaus
 1901 |   1 |   1 |   0 |  0.00 |    0.0 |     0.0 | repaus
 2001 |   1 |   1 |   1 |  0.10 |   10.0 |   330.0 | repaus
 2101 |   7 |   1 |   1 |  0.20 |   20.0 |   660.0 | repaus
 2201 |   1 |   1 |   1 |  0.30 |   30.0 |   990.0 | repaus
 2301 |   1 |   1 |   1 |  0.40 |   40.0 |  1320.0 | repaus
 2401 |   1 |   1 |   1 |  0.50 |   50.0 |  1650.0 | MISCARE
 2501 |   1 |   1 |   1 |  0.60 |   60.0 |  1980.0 | MISCARE
 2601 |   1 |   1 |   1 |  0.70 |   70.0 |  2310.0 | MISCARE
 2701 |   1 |   1 |   1 |  0.80 |   80.0 |  2640.0 | MISCARE
 2801 |   1 |   1 |   1 |  0.90 |   90.0 |  2970.0 | MISCARE
 2901 |   1 |   1 |   1 |  1.00 |  100.0 |  3300.0 | MISCARE
 3001 |   1 |   1 |   1 |  1.00 |  100.0 |  3300.0 | MISCARE
 3101 |   1 |   1 |   1 |  1.00 |  100.0 |  3300.0 | MISCARE
 3201 |   1 |   1 |   1 |  1.00 |  100.0 |  3300.0 | MISCARE
 3301 |   1 |   1 |   1 |  1.00 |  100.0 |  3300.0 | MISCARE
```

Figura 4.2 – Captură de ecran a output-ului în Wokwi

Figura 4.2 reprezintă captura reală din Serial Monitor al rulării în Wokwi, cu timestamp-uri la pas de 100 ms și valorile brut, sat, med, avg, A, V și stare. Ea servește drept dovadă a utilizării simulatorului

Wokwi; proiectul cu circuitul ESP32 este disponibil la adresa web https://wokwi.com/projects/478948111594743809

Tabelul 4.2 prezintă eșantioane reprezentative din captura Wokwi.

Tabelul 4.2 – Eșantioane reprezentative din Serial Monitor (Wokwi)

t, ms	brut	sat	med	avg	A, %	V [mV]	stare
701	0	0	0	0,00	0	0,0	repaus
1201	0	0	0	0,00	0	0,0	repaus
1601	1	1	0	0,00	0	0,0	repaus
1701	0	0	0	0,00	0	0,0	repaus
1901	1	1	0	0,00	0	0,0	repaus
2001	1	1	1	0,10	10	330,0	repaus
2101	7	1	1	0,20	20	660,0	repaus
2301	1	1	1	0,40	40	1320,0	repaus
2401	1	1	1	0,50	50	1650,0	mișcare
2801	1	1	1	0,90	90	2970,0	mișcare
2901	1	1	1	1,00	100	3300,0	mișcare
3301	1	1	1	1,00	100	3300,0	mișcare

În eșantioanele din tabel, impulsul brut de la 1601 ms este filtrat de mediana pe cinci eșantioane (med = 0), fără schimbarea stării sistemului. Valoarea 7 de la 2101 ms reprezintă un test artificial al saturării: este limitată la 1 și, fiind înconjurată de eșantioane HIGH, nu modifică ieșirea medianului.

La 2001 ms mediana devine 1, dar indicele A este de numai 10%, insuficient pentru activarea detecției; mediana trece la 1 cu un eșantion mai devreme decât la un front curat, deoarece impulsul de la 1601 ms se află încă în fereastra de cinci eșantioane. Apoi A crește cu 10% la fiecare eșantion și ajunge la 50% la 2401 ms, când starea se schimbă în „MISCARE". La 2901 ms fereastra de mediere este completată cu valori HIGH, astfel încât A = 100%, iar tensiunea echivalentă calculată ajunge la 3300 mV. Întârzierea de 400 ms dintre med = 1 și A = 50% se observă și în reprezentarea grafică din figura 4.1.

Captura Wokwi prezentată acoperă frontul crescător; comportamentul la golurile LOW din timpul detecției și la frontul descrescător nu este documentat de datele Wokwi din această lucrare și nu este folosit ca rezultat experimental, iar figura 4.1, construită din aceleași date, nu le acoperă nici ea.

Aceste rezultate ilustrează comportamentul filtrului median și al mediei mobile pentru valorile furnizate din simularea Wokwi. Ele nu demonstrează că filtrul elimină alarmele false reale ale senzorului PIR: un impuls fals suficient de lung poate trece prin filtrare și poate activa sistemul. De asemenea, tensiunea V din tabel este o valoare echivalentă calculată din avg, nu o măsurătoare analogică independentă.

CONCLUZII

A fost analizat traseul software al semnalului digital de la ieșirea modulului PIR simulat în Wokwi, citit de ESP32 pe GPIO4. Lanțul limitează valorile la 0…1, aplică filtrul median cu NMED = 5 și media pe NAVG = 10 eșantioane, apoi calculează indicele de activitate A și valoarea echivalentă V.

Datele din Serial Monitor arată că la frontul crescător med devine 1 la 2001 ms, iar pragul A ≥ 50% este atins la 2401 ms, când starea devine „MISCARE". Impulsul izolat de la 1601 ms este eliminat de filtrul median. Valoarea brută 7 de la 2101 ms este un test artificial al saturării și nu poate fi generată de digitalRead() în regim normal; ea este limitată la 1. Întârzierea dintre med = 1 și A = 50% (400 ms) se observă și în reprezentarea grafică din figura 4.1.

Filtrul median N = 5 este potrivit pentru eliminarea glitch-urilor electrice de cel mult două eșantioane, dar nu elimină alarmele false reale ale HC-SR501, al căror timp de menținere este de minimum 0,5 s. Pentru acestea este necesară o măsură suplimentară, precum confirmarea prin două detecții într-un interval stabilit sau impunerea unei durate minime a stării HIGH. În schema Wokwi, GPIO4 este conectat direct la OUT, fără divizor 1 kΩ/2 kΩ; pentru un modul fizic trebuie verificat nivelul electric al ieșirii înainte de conectare.

NOTĂ PRIVIND UTILIZAREA INSTRUMENTELOR AI

La realizarea lucrării a fost utilizat un asistent AI(Claude) pentru sprijin la structurarea raportului, redactarea și revizuirea textului și explicarea implementării software. Informațiile rezultate au fost revizuite, validate și asumate de autor. Autorul a rulat personal proiectul în Wokwi și a verificat datele din Serial Monitor prezentate în figura 4.2 și în tabelul 4.2.

BIBLIOGRAFIE

[1] HANDSONTEC. HC-SR501 PIR Motion Sensor [online]. Fișă tehnică. [citat 25.10.2026]. Disponibil: https://www.handsontec.com/dataspecs/SR501%20Motion%20Sensor.pdf.

[2] FREERTOS. vTaskDelayUntil() [online]. Documentație API. [citat 25.10.2026]. Disponibil: https://freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/02-vTaskDelayUntil.

[3] PLATFORMIO. PlatformIO IDE [software]. [citat 25.10.2026]. Disponibil: https://platformio.org/.

[4] WOKWI. Wokwi Online ESP32 Simulator [software]. [citat 25.10.2026]. Disponibil: https://wokwi.com/.

ANEXE

ANEXA A.

Codul sursă main.cpp

```
/*
 * L2 - Achizitia semnalului si conditionarea software
 * Platforma : ESP32 (PlatformIO, framework Arduino, FreeRTOS)
 * Senzor    : modul PIR (prototipul din L1), iesirea OUT citita digital pe GPIO4
 * Lant      : citire -> saturare -> filtru median -> mediere -> conversie
 */
#include <Arduino.h>

/* ---------------- Parametri configurabili ---------------- */
#define PIN_PIR         4      /* GPIO4: iesirea OUT a modulului PIR (digitala) */
#define TS_MS           100    /* perioada de esantionare [ms] */
#define N_MED           5      /* fereastra filtrului median (numar impar) */
#define N_AVG           10     /* fereastra filtrului de mediere */
#define LVL_MIN         0      /* nivel LOW al iesirii PIR (fisa tehnica: 0 V) */
#define LVL_MAX         1      /* nivel HIGH al iesirii PIR (fisa tehnica: 3,3 V) */
#define V_HIGH_MV       3300   /* tensiunea nivelului HIGH [mV] */
#define A_PRAG          50.0f  /* prag de decizie pentru indicele de activitate [%]
*/
#define SIM_DISTURBANCE 1      /* 1 = suprapune perturbatii de test pe semnal */

#if (N_MED % 2) == 0
#error "N_MED trebuie sa fie impar"
#endif

/* ---------------- Starea filtrelor ---------------- */
static int  medBuf[N_MED];
static int  medIdx  = 0;
static bool medInit = false;

static int  avgBuf[N_AVG];
static long avgSum  = 0;
static int  avgIdx  = 0;
static bool avgInit = false;

/* ---------------- Citire bruta ---------------- */
static int readRaw(void)
{
    return digitalRead(PIN_PIR);           /* 0 sau 1, fara prelucrare */
}

/* ---------------- Perturbatii de test (doar pentru validare) ---------------- */
#if SIM_DISTURBANCE
static int injectDisturbance(int raw, uint32_t k)
{
    uint32_t p = k % 100u;
    if (p == 10)              raw = 1;     /* impuls HIGH izolat, 1 esantion */
    if (p == 15)              raw = 7;     /* citire invalida (in afara 0...1) */
    if (p == 20 || p == 21)   raw = 1;     /* impuls HIGH de 2 esantioane */
    if (p == 45)              raw = 0;     /* golire LOW de 1 esantion */
    if (p == 55 || p == 56)   raw = 0;     /* golire LOW de 2 esantioane */
    if (p >= 62 && p <= 64)   raw = 0;     /* rafala LOW de 3 esantioane */
    return raw;
}
#endif
```

```
/* ---------------- Saturare ---------------- */
static int saturate(int x)
{
    if (x < LVL_MIN) return LVL_MIN;
    if (x > LVL_MAX) return LVL_MAX;
    return x;
}

/* ---------------- Filtru median (fereastra alunecatoare N_MED) ---------------- */
static int medianFilter(int x)
{
    if (!medInit) {                         /* initializare cu primul esantion */
        for (int i = 0; i < N_MED; i++) medBuf[i] = x;
        medInit = true;
    }
    medBuf[medIdx] = x;
    medIdx = (medIdx + 1) % N_MED;

    int tmp[N_MED];
    for (int i = 0; i < N_MED; i++) tmp[i] = medBuf[i];
    for (int i = 1; i < N_MED; i++) {       /* sortare prin insertie */
        int v = tmp[i], j = i - 1;
        while (j >= 0 && tmp[j] > v) { tmp[j + 1] = tmp[j]; j--; }
        tmp[j + 1] = v;
    }
    return tmp[N_MED / 2];
}

/* ---------------- Filtru de mediere (medie aritmetica, N_AVG) ---------------- */
static float averageFilter(int x)
{
    if (!avgInit) {
        for (int i = 0; i < N_AVG; i++) avgBuf[i] = x;
        avgSum = (long)x * N_AVG;
        avgInit = true;
    }
    avgSum += x - avgBuf[avgIdx];
    avgBuf[avgIdx] = x;
    avgIdx = (avgIdx + 1) % N_AVG;
    return (float)avgSum / N_AVG;
}

/* ---------------- Conversie in marime fizica ---------------- */
static float averageToActivity(float avg)
{
    return 100.0f * (avg - LVL_MIN) / (LVL_MAX - LVL_MIN);   /* indice A [%] */
}

static float activityToMillivolts(float a)
{
    return a / 100.0f * V_HIGH_MV;          /* tensiune medie a iesirii [mV] */
}

/* ---------------- Task de achizitie ---------------- */
static void taskAcquire(void *pv)
{
    TickType_t xLastWake = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(TS_MS);
    uint32_t k = 0;

    printf("%8s | %4s | %4s | %4s | %5s | %6s | %7s | %s\n",
           "t[ms]", "brut", "sat", "med", "avg", "A[%]", "V[mV]", "stare");
```

```
    for (;;) {
        vTaskDelayUntil(&xLastWake, xPeriod);   /* perioada constanta */
        uint32_t t = millis();

        int raw = readRaw();
#if SIM_DISTURBANCE
        raw = injectDisturbance(raw, k);
#endif
        int   sat = saturate(raw);
        int   med = medianFilter(sat);
        float avg = averageFilter(med);
        float act = averageToActivity(avg);
        float mv  = activityToMillivolts(act);

        printf("%8lu | %4d | %4d | %4d | %5.2f | %6.1f | %7.1f | %s\n",
               (unsigned long)t, raw, sat, med, avg, act, mv,
               (act >= A_PRAG) ? "MISCARE" : "repaus");
        k++;
    }
}

void setup()
{
    Serial.begin(115200);
    pinMode(PIN_PIR, INPUT);
    xTaskCreatePinnedToCore(taskAcquire, "acq", 4096, NULL, 2, NULL, 1);
}

void loop()
{
    vTaskDelete(NULL);                      /* loop() nu este folosit */
}
```

ANEXA B

Scriptul Python pentru Figura 4.1

```
import matplotlib.pyplot as plt

# Date extrase din Serial Monitor
data = [
    (701,  0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (801,  0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (901,  0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1001, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1101, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1201, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1301, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1401, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1501, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1601, 1, 1, 0, 0.00, 0.0,   0.0, "repaus"),
    (1701, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1801, 0, 0, 0, 0.00, 0.0,   0.0, "repaus"),
    (1901, 1, 1, 0, 0.00, 0.0,   0.0, "repaus"),
    (2001, 1, 1, 1, 0.10, 10.0, 330.0, "repaus"),
    (2101, 7, 1, 1, 0.20, 20.0, 660.0, "repaus"),
    (2201, 1, 1, 1, 0.30, 30.0, 990.0, "repaus"),
    (2301, 1, 1, 1, 0.40, 40.0, 1320.0, "repaus"),
    (2401, 1, 1, 1, 0.50, 50.0, 1650.0, "MISCARE"),
    (2501, 1, 1, 1, 0.60, 60.0, 1980.0, "MISCARE"),
    (2601, 1, 1, 1, 0.70, 70.0, 2310.0, "MISCARE"),
```

```
    (2701, 1, 1, 1, 0.80, 80.0, 2640.0, "MISCARE"),
    (2801, 1, 1, 1, 0.90, 90.0, 2970.0, "MISCARE"),
    (2901, 1, 1, 1, 1.00, 100.0, 3300.0, "MISCARE"),
    (3001, 1, 1, 1, 1.00, 100.0, 3300.0, "MISCARE"),
    (3101, 1, 1, 1, 1.00, 100.0, 3300.0, "MISCARE"),
    (3201, 1, 1, 1, 1.00, 100.0, 3300.0, "MISCARE"),
    (3301, 1, 1, 1, 1.00, 100.0, 3300.0, "MISCARE")
]

t_sec = [row[0] / 1000.0 for row in data]
raw = [row[1] for row in data]
med = [row[3] for row in data]
act = [row[5] for row in data]

# Dimensiune stabila pentru ecran si Word
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8.5, 5.0), sharex=True, dpi=300)

# --- Subplot 1: Semnal brut vs Median ---
ax1.plot(t_sec, raw, color='#666666', linewidth=1.5, linestyle='--', marker='o',
markersize=4, label='Brut', drawstyle='steps-post')
ax1.plot(t_sec, med, color='#D32F2F', linewidth=2.2, marker='s', markersize=4,
label='Filtru Median', drawstyle='steps-post')

# Limita verticala cu spatiu suficient sus
ax1.set_ylim(-0.15, 1.7)
ax1.set_yticks([0, 1])
ax1.set_ylabel('Stare binara', fontsize=10)
ax1.grid(True, linestyle=':', alpha=0.6)

# Mutam legenda in dreapta sus
ax1.legend(loc='upper right', framealpha=0.9, fontsize=8.5)

# Adnotari pozitionate clar cu fundal alb protector
bbox_props = dict(boxstyle="round,pad=0.2", fc="white", ec="none", alpha=0.8)

ax1.annotate('Glitch 1 (filtrat)', xy=(1.6, 1.02), xytext=(1.0, 1.45),
             arrowprops=dict(arrowstyle='->', color='black', lw=1.0),
             fontsize=8.5, fontweight='bold', bbox=bbox_props)

ax1.annotate('Raw=7 (saturat)', xy=(2.1, 1.02), xytext=(2.0, 1.45),
             arrowprops=dict(arrowstyle='->', color='black', lw=1.0),
             fontsize=8.5, fontweight='bold', bbox=bbox_props)

# --- Subplot 2: Indicele de activitate A [%] ---
start_m = None
for t, a in zip(t_sec, act):
    if a >= 50 and start_m is None:
        start_m = t
if start_m is not None:
    ax2.axvspan(start_m, t_sec[-1], color='#E8F5E9', alpha=0.7, label='Stare:
MISCARE')

ax2.plot(t_sec, act, color='#1976D2', linewidth=2.2, marker='o', markersize=4,
label='Indice A [%]')
ax2.axhline(50, color='#D32F2F', linestyle=':', linewidth=1.2, label='Prag de decizie
(50%)')

ax2.set_ylabel('A [%]', fontsize=10)
ax2.set_xlabel('Timp [s]', fontsize=10)
ax2.set_ylim(-10, 125)
ax2.set_yticks([0, 25, 50, 75, 100])
```

```
ax2.grid(True, linestyle=':', alpha=0.6)

# Mutam legenda in stanga sus fara sa suprapuna textul
ax2.legend(loc='upper left', framealpha=0.9, fontsize=8.5)

# Adnotare comutare starii
ax2.annotate('Comutare la MISCARE (2.4s)', xy=(2.4, 50), xytext=(1.45, 15),
             arrowprops=dict(arrowstyle='->', color='#1976D2', lw=1.2),
             fontsize=8.5, fontweight='bold', color='#1976D2', bbox=bbox_props)

plt.tight_layout()
plt.savefig("Figura_4_1_Perfect.png", dpi=300, bbox_inches='tight')
plt.show()
```